

STUDENT ACADEMIC TRACKER

Comprehensive Technical Project & Software Architecture Report

Author / Developer: Rudra Pratap Singh

Academic Supervisor: Mr. Dheresh Soni

Course / Subject: Programming in Python

Implementation: Standalone CLI System

1. FORMAL ACKNOWLEDGEMENT

*"I express my deepest gratitude to my esteemed Python professor, **Mr. Dheresh Soni**, whose expert guidance, inspiring lectures, and continuous encouragement were instrumental in the conceptualization and completion of this project. His emphasis on modular code architecture, robust exception handling, and standard library efficiency provided the foundation for building this software. I am sincerely thankful for his mentorship throughout this academic endeavor."*

2. EXECUTIVE SUMMARY

The **Student Academic Tracker & Grade System** is a lightweight, menu-driven Command Line Interface (CLI) application engineered in pure Python. Designed as a standalone academic management utility, it enables educational institutions and individual administrators to manage student records, record course evaluations, compute exact credit-weighted Cumulative Grade Point Averages (CGPA), generate formatted ASCII transcripts, and extract batch performance metrics without external database dependencies.

3. PROJECT OBJECTIVES

The primary technical goals achieved during the development of this application include:

- Zero-Dependency Engineering:** Utilizing exclusively standard Python libraries (json, csv, os, sys) to maximize portability across system environments.
- Robust Local Persistence:** Implementing a dual-layer data handler featuring local JSON database persistence and structured CSV export capabilities.
- Automated Evaluation Logic:** Eliminating manual calculation errors by implementing standard grade-point conversion tables and weighted CGPA algorithms.
- Input Sanitization & Fault Tolerance:** Preventing runtime crashes through robust input validation loops that catch type mismatches, empty values, and out-of-range inputs.

4. MATHEMATICAL MODEL & GRADING MECHANICS

The academic evaluation module uses a standardized 10-point relative scale. Raw numerical scores (\$0 - 100\$) entered into the system are mapped to specific letter grades and numeric grade points before being factored into the CGPA calculation engine.

Marks Range (%)	Letter Grade	Grade Points (\$P_i\$)	Performance Assessment
90.0 – 100.0	S	10.0	Outstanding
80.0 – 89.9	A	9.0	Excellent
70.0 – 79.9	B	8.0	Very Good
60.0 – 69.9	C	7.0	Good
50.0 – 59.9	D	6.0	Satisfactory
40.0 – 49.9	E	5.0	Pass
Below 40.0	F	0.0	Fail / Re-examine

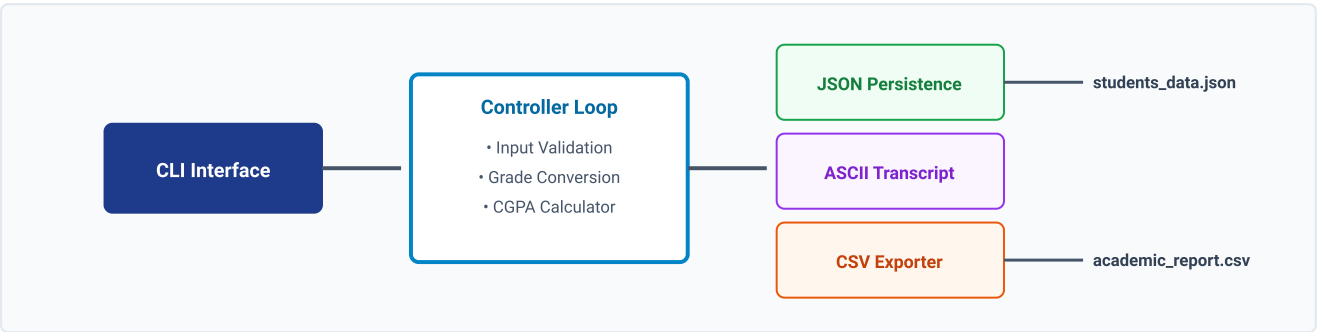
WEIGHTED CGPA MATHEMATICAL FORMULA

$$CGPA = \Sigma (C_i \times P_i) / \Sigma C_i$$

Where C_i represents Course Credits ($1 \leq C_i \leq 6$) and P_i represents Earned Grade Points.

5. SYSTEM ARCHITECTURE & COMPONENT DIAGRAM

The application follows a clean modular controller-view architecture designed around an infinite operational loop controlled by user prompts and local file handlers.



6. FUNCTIONAL MODULE SPECIFICATION

The software source is structured into functional procedures, separating user interaction, calculation routines, and file I/O operations.

Function Unit	Input Parameters	Operational Responsibility
load_database()	None	Reads and parses students_data.json. Handles file missing or corrupt errors cleanly.
save_database()	data (dict)	Serializes the main dictionary to disk in formatted JSON structure.
convert_marks_to_grade()	marks (float)	Maps raw numerical scores to letter grades and numerical grade points.
compute_student_gpa()	courses (dict)	Iterates course records to compute weighted CGPA and total earned credits.
render_transcript()	db (dict)	Formatted ASCII transcript output displaying student info and completed courses.
export_csv_report()	db (dict)	Generates academic_report.csv report containing cohort metrics.
show_analytics()	db (dict)	Calculates cohort average GPA, top performer, and lowest performer.

7. TERMINAL OUTPUT & TRANSCRIPT RENDERING SAMPLE

The application formats student academic histories into formatted ASCII transcripts suitable for terminal viewing or printing.

```
===== ACADEMIC TRANSCR
```

8. BATCH ANALYTICS & INTELLIGENCE ENGINE

The system provides administrators with macro-level insight into class performance through its integrated analytics processor.

Cohort GPA Averaging
Computes overall batch mean GPA across all active students with completed grade entries.

High-Performer Identification
Automatically flags top-tier academic achievements for honors or commendation listing.

Academic Risk Monitoring
Identifies students falling below minimum academic progress thresholds for counseling.

CSV Metric Export
Outputs clean, structured tabular datasets for integration with MS Excel or visualization tools.

9. ERROR HANDLING & QUALITY ASSURANCE TESTING

To guarantee stability under user operational errors, input routines were validated against common fault conditions:

- **Type Mis-matching:** Wrapped numerical prompts inside try-except ValueError blocks to prevent program termination on string inputs.
- **Boundary Enforcement:** Enforced strict value limits ($0 \leq \text{Marks} \leq 100$ and $1 \leq \text{Credits} \leq 6$).
- **Duplicate Prevention:** Validates registration numbers prior to creation to prevent overwriting existing student records.
- **Graceful Interrupts:** Trapped KeyboardInterrupt (Ctrl+C) signals to ensure clean program termination without tracebacks.

10. CONCLUSION & FUTURE ROADMAP

The **Student Academic Tracker & Grade System** successfully delivers a lightweight, reliable, and fault-tolerant solution for student record management. By eliminating external dependencies and implementing standard Python design patterns under the guidance of **Mr. Dheresh Soni**, the software achieves fast execution and full cross-platform compatibility.

Future Enhancements: Future iterations will aim to integrate a Graphical User Interface (GUI) using tkinter, support database integration via sqlite3, and enable PDF transcript exporting capabilities.

Developer: Rudra Pratap Singh

Supervisor: Mr. Dheresh Soni

Status: Completed & Verified